

Correction-Aware Data Admission: Candidate-Bound Authorization and Field-Level Provenance for AI-Derived Updates

Liang Hanghao^a, Xuan Wentao^a, Chen Qile^a, Peng Peng^{a,*}

^a*College of Computer Science and Electronic Engineering, Hunan University, Lushan South Road, Yuelu District, Changsha, 410082, China*

Abstract

AI-assisted data acquisition separates a machine proposal from the value a human authorizes for a database. Identical final values can conceal different reviewed candidates, corrected values, and field sources. We specify a correction-aware admission relation that binds an exact persisted candidate to an explicit authorized value, checks predecessor freshness, and atomically constructs a complete successor with total field-level provenance. Five paired histories establish class-wise irredundancy under declared observations. A Python/SQLite realization connects the relation to bounded analysis, persisted-state projections, and a 35-case fault catalogue. A constructed comparison evaluates a context-preserving event journal, its exact-binding extension, and the reference implementation. The two exact-binding mechanisms agree on all 165 corresponding cases. The context journal admits 15 equal-valued candidate substitutions that satisfy its value/context policy but violate instance accountability; the stricter mechanisms reject those substitutions. In 60 scripted browser cases, a server-held review-session gate blocks nine request substitutions accepted by the original path. Both paths accept three display-only substitutions, locating the trusted presentation boundary. Current-version workloads characterize admission, provenance reconstruction, and storage costs; bulk and point trace access return identical complete objects. The results connect the admission relation to enforceable integrity obligations across two storage designs, within the declared model, trust boundary, and measured workloads.

*Corresponding author. Email address: hnu16pp@hnu.edu.cn

Keywords: data integrity, data provenance, human authorization,
AI-assisted data acquisition, transactional admission, relational verification

1. Introduction

An AI-assisted acquisition system may extract a value, propose an update, and ask a person to approve it. Once admitted, the value can drive queries and decisions. Its authority depends on a history that the final value alone cannot identify: which persisted proposal was reviewed, what value was authorized, and which predecessor and field sources the resulting record retained.

Consider quantity 90. A recognizer proposes 100, but a reviewer authorizes 101; an unchanged batch-code field should keep its earlier source. Overwriting the proposal with 101 erases the machine–human distinction. Recording only 101 and a reviewer leaves the reviewed proposal ambiguous. Copying values without sources produces a numerically correct snapshot with incomplete provenance. Correction-aware admission must retain the proposal x_c separately from the authorized value x_a , including

$$x_c \neq x_a. \tag{1}$$

Candidate identity gives a sharper separator. Two persisted extraction attempts may propose the same value for the same field and evidence. A value/context policy can treat them as interchangeable. An instance-accountability policy must distinguish the candidate selected by the reviewer from an equal-valued substitute. We specify that stricter relation and evaluate both policies explicitly.

Transactions, optimistic validation, and provenance provide established foundations (Gray, 1981; Kung and Robinson, 1981; Moreau and Missier, 2013). The contribution is a correction-aware admission relation over their semantic objects:

$$\begin{aligned} \textit{exact candidate} &\rightarrow \textit{candidate-bound authorization} \\ &\rightarrow \textit{explicit authorized value} \\ &\rightarrow \textit{complete successor and total field sources.} \end{aligned} \tag{2}$$

The candidate remains historical evidence; authorization supplies permission for the committed value. One transaction creates the successor, assigning new transitions to changed fields and preserving exact sources for unchanged fields. Relational tables and event journals can both represent these obligations.

Five information classes characterize the histories the relation distinguishes: candidate identity and context, value roles, freshness, successor completeness, and field-source attribution. For each class, a paired construction has different authority outcomes but equal remaining observations when that class is omitted. The result establishes class-wise irredundancy within the declared history and observation model.

We realize the relation in AUTO-DECTE, a Python/SQLite service, and make three contributions:

- C1: A correction-aware admission contract and conditional characterization.** Separate proposal and authorized values, candidate-bound review, and complete field sources support five failure-distinguishing information classes.
- C2: Relational and transactional enforcement.** An Alloy model, in-transaction validation, version compare-and-swap, selected formal-concrete projections, and persisted-state checks connect the relation to executable behavior.
- C3: Comparative, review-boundary, and cost evidence.** Event-journal controls isolate exact candidate binding; browser controls locate the transfer of review intent; current-version workloads measure admission, tracing, and storage.

The central comparison has 165 corresponding constructed cases per mechanism. The exact-binding event journal and reference agree on every decision and provenance-query answer. The context journal accepts 15 equal-valued substitutions that satisfy its value/context policy but violate instance accountability. These are also 15 additional rejections by the stricter mechanisms under the looser policy. The separator therefore identifies the authorization relation rather than a uniquely capable storage architecture.

Review intent must reach that relation faithfully. A server-held review session rejects nine request substitutions accepted by the original path, while both paths accept three display-only substitutions. The contract enforces the authorization supplied to it; authenticated identity and faithful review presentation remain trust assumptions. Figure 1 summarizes this boundary.

Same values, different admission histories

Preserve the reviewed proposal, authorized correction, and every successor field source.

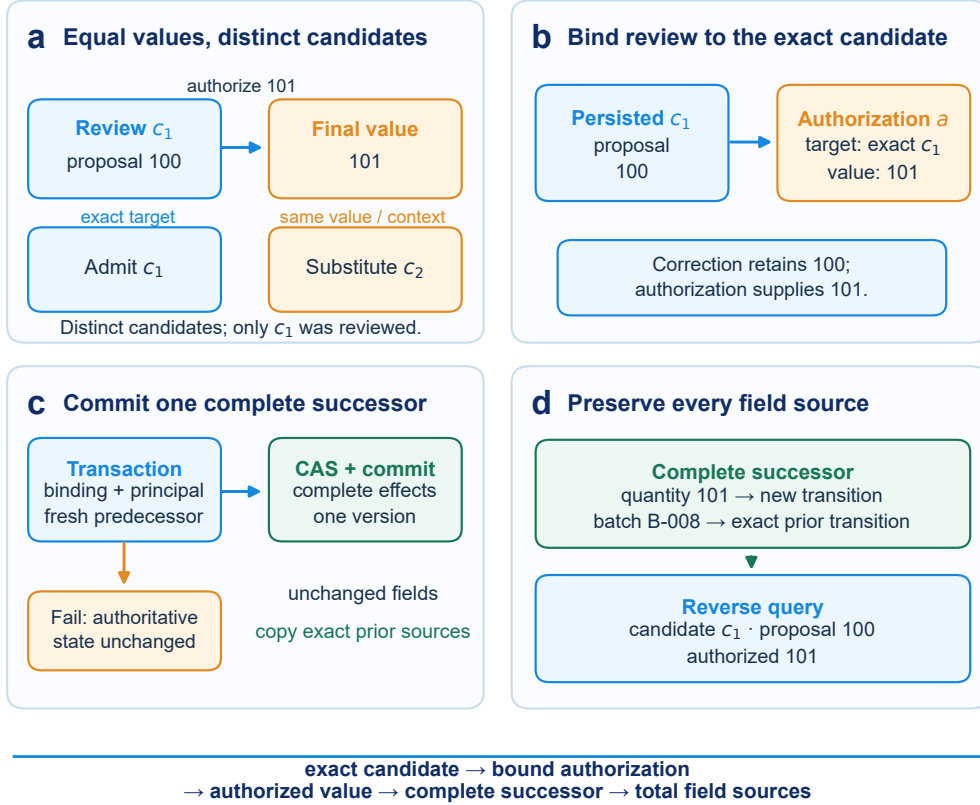


Figure 1: Correction-aware admission: (a) equal-valued candidates remain distinct under instance authorization; (b) Correction retains the original proposal; (c) failed admission leaves authoritative state unchanged; (d) changed and unchanged fields retain their respective sources. Values illustrate a constructed example. OpenAI Codex (GPT-6) and OpenAI’s image-generation tool assisted visual design on 27 September 2026; the latter exposed no model/version identifier. The checked vector diagram was rendered with Matplotlib 3.10.8.

2. Background and related work

2.1. *Transactions, provenance, and event histories*

Transactions provide atomic, durable state transformations; optimistic concurrency control validates conflicting work before installing effects (Gray, 1981; Kung and Robinson, 1981). Our compare-and-swap is an application freshness check inside that transaction. The admission relation specifies which candidate, authorization, value, and source relationships the transaction must enforce. Existence of a candidate or a current predecessor alone does not determine whether an approval targets that candidate or its separately authorized correction.

Why-, how-, and where-provenance describe the reasons, derivations, and origins of query results (Buneman et al., 2001; Cheney et al., 2009). PROV-DM represents entities, activities, agents, derivation, and responsibility, with application extension points (Moreau and Missier, 2013). Our field-source map identifies durable transitions responsible for a committed record’s fields. A proposal can be a PROV entity and review an activity; the admission relation adds the operational obligation to check their bindings before creating a complete snapshot.

Transaction reenactment reconstructs multi-version provenance from update histories, audit logs, and prior database versions (Arab et al., 2018). Such histories can include proposal and review data when the application records them. Our focus is the correction-aware association to enforce: the immutable proposal, the explicit human-authorized value, and the source of each changed or copied-forward field.

Event sourcing records a sequence of changes from which state can be reconstructed (Fowler, 2005). Its schema and validation rules determine the granularity of approval. This motivates our two rich-context journals: both retain versions, sources, and transactional behavior, while one additionally binds the exact reviewed instance. The exact journal’s agreement with the reference supports enforcement in both representations. Equal-valued substitution exposes the policy distinction between approved content and approved candidate instances.

2.2. *Adjacent activation and governed-memory contracts*

Recent preprints address neighboring authority boundaries. Continuity Kernel binds proposal identity, exact predecessor, pre-state authority, evidence, lineage, and a complete accepted unit, with relational, object-manifest, and

replicated-log realizations (He and Yu, 2026a). General proposal/authority separation and complete-successor activation are therefore established in that account. Our contribution concerns a reviewed field candidate whose proposed value can differ from the authorized correction, exact field-source propagation, and the conditional observation analysis and executable evidence for that relation.

MemTxn separates source-support admission, temporal visibility, and application-state recovery (Cui et al., 2026); its source-support predicate complements the candidate-to-human-authorization predicate studied here. MutMem binds signed retrieval-weight changes to provenance, signer epochs, old/new weights, and predecessor continuity (Saidi, 2026). Its mutation and cryptographic assurance differ from reviewed field correction. LatticeMind retains claim status, conflict, supersession, and provenance (Zhou et al., 2026). Stored Is Not Supported begins with an authenticated accepted head and governs subsequent assertion release (He and Yu, 2026b). We compare these preprints’ stated scopes; their representations may encode the relation evaluated here.

2.3. Review binding and executable analysis

OAuth DPoP specifies proof-of-possession and request method/URI bindings (Fett et al., 2023). Our review experiment similarly distinguishes what a request binds from what a reviewer was shown. Host authentication and channel integrity remain premises. AgentDojo evaluates tool agents over untrusted data with separate task and security outcomes (Debenedetti et al., 2024). Our new studies instead examine deterministic admission and review boundaries using archived values and constructed histories; historical model integration is reported separately.

Alloy and Kodkod provide SAT-based bounded relational analysis (Jackson, 2002; Torlak and Jackson, 2007); TestEra connects relational specifications and executable programs through abstraction and concretization (Marinov and Khurshid, 2001). We apply these methods to legal histories, conjunct removals, and selected persisted-state projections. The paired-history argument characterizes observations, Alloy checks its encoding within finite scopes, and concrete tests examine persisted behavior. These evidence layers support distinct parts of the admission relation.

3. Authoritative-state admission contract

3.1. Candidate, authorization, and authoritative state

Correction-aware admission links an untrusted persisted candidate c , attributable authorization a , and authoritative pre-state Σ_v :

$$\text{Admit}(c, a, \Sigma_v) = \Sigma_{v+1}. \quad (3)$$

The candidate stays non-authoritative historical evidence. Proposal origin, value authority, and committed value remain distinct roles.

Let r be a record with a nonempty authoritative field set F_r . Its committed state Σ_v contains a total value map $V_v : F_r \rightarrow X$ and a total source map $S_v : F_r \rightarrow T$, where T is the set of durable fact transitions. A machine candidate for field f is

$$c = \langle r, f, e, v_{\text{exp}}, x_c, p, c_{\text{id}} \rangle, \quad (4)$$

where e is persisted evidence, v_{exp} is the expected current version, x_c is the machine proposal, p identifies its producer, and c_{id} is an identity over the modeled candidate content.

Evidence identity combines a content digest and canonical record/field locator:

$$EID(e) = \langle H(\text{bytes}(e)), L(r, f, u, \kappa) \rangle. \quad (5)$$

Here H is SHA-256, and L contains the owner record, field, normalized storage-relative URI u , locator version, and optional crop κ . Alloy abstracts content and locators as primitive domains. Identity uniqueness in its scopes relies on collision resistance and canonicalization/store integrity in the implementation (Supplement S2).

An attributable authorization is

$$a = \langle c_{\text{id}}, x_a, q \rangle, \quad (6)$$

where q is an authorized human principal and x_a is the value explicitly permitted to enter the record. The dual-value semantics are

$$\begin{aligned} \text{proposal}(c) &= x_c, & \text{authorize}(a) &= x_a, \\ \text{commit}(\Sigma_{v+1}, f) &= x_a, & \text{origin}(\Sigma_{v+1}, f) &= c. \end{aligned} \quad (7)$$

Accept has $x_c =_{\text{json}} x_a$. Correction has $x_c \neq_{\text{json}} x_a$; the candidate remains unchanged and the committed value derives its authority from a . Here origin identifies the retained proposal, not the authority for a corrected value.

Canonical value equality. Write $x =_{\text{json}} y$ when sorted-key, whitespace-minimal JSON serializations agree. This distinguishes 2 from 2.0 and 1 from `true`, although Python equality does not. Change detection, value checks, tracing, the independent oracle, and projection use this relation. Section 6 reports the cross-layer repair.

3.2. History, observations, and outcomes

An authority-relevant history is

$$h = \Sigma_0 \xrightarrow{e_1} \Sigma_1 \xrightarrow{e_2} \dots \xrightarrow{e_n} \Sigma_n, \quad (8)$$

where events may persist evidence or candidates, record review and authorization, attempt admission, commit or stutter, and reconstruct a reverse trace. The abstraction describes semantic information rather than requiring particular table or column names.

We group the observable distinctions into

$$\mathcal{D} = \{D_C, D_V, D_F, D_B, D_S\}, \quad (9)$$

where D_C is candidate identity and context, D_V is dual-value attribution, D_F is freshness, D_B is batch/successor completeness, and D_S is total source attribution. For $I \subseteq \mathcal{D}$, $\pi_I(h)$ retains only the information classes in I . Candidate identity may be implemented by a certificate, stable handle, or another equivalence class, provided it does not collapse to value equality. Freshness may likewise use a version, state hash, epoch, or comparable discriminator.

To make “retains” precise, let $\text{obs}_d(h)$ be the observation function for class d and define

$$\pi_I(h) = \langle \text{obs}_d(h) \rangle_{d \in I}. \quad (10)$$

The functions are logical projections, not claims that a database must store five physically separate objects. A certificate can co-locate several classes. In a projection, however, a content address is treated as an opaque equality-preserving handle: its preimage is not available as a side channel. This prevents a nominally omitted value or version from being recovered merely because the concrete certificate hash covers it. The complete observation functions are given in Supplement S2.

The full-history authority-outcome label is

$$O(h) = \langle \text{status}, \text{successor}, \text{trace} \rangle, \quad (11)$$

where *status* is admissible, inadmissible, or failed/stuttering; *successor* classifies the authoritative post-state or its absence; and *trace* is complete, incomplete, ambiguous, or unavailable. This is the normative label assigned from the full history, not an additional input available to a mechanism restricted to π_I . A binary accept/reject label is insufficient because value-identical successors may differ in source completeness.

An observation set I fails to distinguish a failure family when there exist a safe history h_s and unsafe history h_u such that

$$O(h_s) \neq O(h_u) \quad \wedge \quad \pi_I(h_s) = \pi_I(h_u). \quad (12)$$

Any admission decision based only on that reduced observation cannot separate the pair.

The executable histories contain candidate and authorization registries, declared items, expected and observed versions, a connected commit sequence, endpoint maps, and transition sources. Projections and outcomes are derived from these relations. Schema fields determine total map domains; declared items determine required effects. Missing or conflicting sources remain failures in the domain. Supplement S2 gives the complete functions, finite tuple, and checker rules.

3.3. Conditional failure-distinguishability characterization

Table 1 summarizes the paired histories for this fixed observation basis.

Proposition 1 (conditional failure distinguishability). For each information class $d_i \in \mathcal{D}$, the declared failure model contains a safe history and a corresponding unsafe history whose authority outcomes differ but whose projections become equal when d_i is omitted. Consequently, an admission mechanism that observes only $\mathcal{D} \setminus \{d_i\}$ cannot distinguish the paired failure family under this model.

Proof sketch (by construction). For D_C , hold the value-role, temporal, batch, and source observations constant while changing non-temporal candidate context. For D_V , retain the same candidate, proposal value 100, and committed value 101, but falsely attribute the corrected value to the machine proposal. For D_F , change only the admission-entry freshness comparison. For D_B , compare one two-field commit with two connected single-field commits that reach the same final values and sources; the latter exposes an intermediate partial state. Effect domains and successor counts follow from these sequences. For

Table 1: Failure-distinguishing information classes in the declared observation model.

Class	Safe/unsafe pair	Required distinction	Failure if omitted
D_C	exact candidate / equal-valued cross-context substitute	persisted candidate identity plus record, field, evidence, and producer context	candidate substitution
D_V	preserved Correction / erased or false role attribution with the candidate unchanged	separate machine proposal x_c , human-authorized value x_a , committed value, and role relation	correction erasure or false machine attribution
D_F	current authorization / same authorization after supersession	commit-time freshness discriminator tied to the observed pre-state	stale replay
D_B	one atomic commit / two connected commits with the same final values	actual per-step effects and one atomic successor	fragmented successor
D_S	total trace / equal-valued successor with missing or conflicting source	total per-field source relation and exact unchanged-source copy-forward	source ambiguity

D_S , remove one source anchor while retaining candidates, values, freshness, and the commit sequence. Each construction changes its selected observation coordinate and the derived outcome, while leaving the other four coordinates equal.

Thus, for every $d_i \in \mathcal{D}$, the constructed histories satisfy

$$\pi_{\mathcal{D} \setminus \{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D} \setminus \{d_i\}}(h_u^{(i)}) \quad \text{and} \quad O(h_s^{(i)}) \neq O(h_u^{(i)}). \quad (13)$$

Any decision function of the reduced projection receives the same input for both histories and cannot assign both distinct required outcomes. This establishes the claimed conditional irredundancy.

The executable checker validates each complete $\hat{h}$ and derives observations and outcomes from its relations; witnesses store neither. It checks domain validity, unequal outcomes, equal four-class projections, and unequal full projections. Schema fields define the total value/source domains, whereas declared item fields define the required effect domain.

The original D_C pair changes record context while retaining the candidate handle, establishing irredundancy of the combined class. An additional pair fixes the two-candidate registry and every context/value component, then selects either the authorized candidate or the equal-valued alternative. It isolates candidate binding within this observation model. A single-field copy-forward control checks the distinction between schema and effect domains. Supplement S2 identifies the executable constructions and raw histories.

Scope of the result. Proposition 1 establishes class-wise irredundancy for the declared failure, history, observation, and outcome model, not universal minimality, schema uniqueness, or completeness over all admission failures. The checker validates the five class pairs, identity pair, and copy-forward control; it is not a general admission oracle. These constructions establish Eq. (13); the separate Alloy checks test encoded assertions and conjunct-removal sensitivity in finite scopes.

3.4. Batch admission relation

A batch B is a nonempty set of items sharing one target record, principal, expected pre-version, and successor. Under Full, item fields are unique. Let

$$\Delta(V_v, V_{v+1}) = \{f \in F_r \mid V_v(f) \neq_{\text{json}} V_{v+1}(f)\}. \quad (14)$$

For a post-initial concrete admission,

$$\Delta(V_v, V_{v+1}) = \{i.f \mid i \in B\} = \{t.f \mid t \in T_{v+1}\}, \quad (15)$$

with one transition per item. At the first certificate-era version, the batch covers all of F_r , establishing total value and source domains. Deletion is outside the model.

For every $i \in B$, admission requires

$$\begin{aligned} i.r &= r, & i.f &\in F_r, \\ EID(i.e) &= EID(i.c.e), & i.v_{\text{exp}} &= v, \\ i.a.c_{\text{id}} &= i.c.c_{\text{id}}, & i.a.q &= q, \\ i.x &=_{\text{json}} i.a.x_a, & t_i.x &=_{\text{json}} V_{v+1}(i.f) =_{\text{json}} i.x. \end{aligned} \quad (16)$$

Together these instantiate the admission preconditions

$$\begin{aligned} &\text{Bind}(a, c) \wedge \text{Fresh}(c, \Sigma_v) \wedge \text{Authorized}(a) \\ &\quad \wedge \text{Complete}(\Sigma_{v+1}) \wedge \text{Attributed}(\Sigma_{v+1}). \end{aligned} \quad (17)$$

The successor is atomic: all admission-time decision, authorization, transition, version, source-map, and audit effects commit together, or authoritative state stutters. This does not require rollback of evidence, candidates, or review intent persisted before the admission attempt. For a changed field, $S_{v+1}(f) = t_i$. For an unchanged field, value and exact source are copied forward:

$$f \notin \Delta(V_v, V_{v+1}) \Rightarrow V_{v+1}(f) =_{\text{json}} V_v(f) \wedge S_{v+1}(f) = S_v(f). \quad (18)$$

Alloy permits same-value admission for correspondence with the historical singleton model; concrete post-initial admissions require a value change. For submitted items U , the admitted batch is $B = \{i \in U \mid i.x \neq_{\text{json}} V_v(i.f)\}$, with $B \neq \emptyset$. A mixed submission admits exactly B and copies unchanged items' exact prior sources into the complete successor, creating no transition or authorization binding for $U \setminus B$. Thus Eq. (15) concerns admitted effects, not all submitted reviews. Recording renewed review or re-attribution without a value change requires a separate review-event path outside this contract; persisted candidate identity allows that path to distinguish equal-valued proposals.

3.5. Operational properties and trust boundary

Model output and candidate-only callers are untrusted. The trusted base comprises the admission service, adapter, exercised transaction semantics, canonicalization and hashing, and principal policy. Host authentication, session integrity, and faithful review presentation are external assumptions. The confirmation API constructs its binding from caller-supplied candidate and value; the contract preserves that binding, not factual truth or protection from a compromised host.

Authorization timing and revocation. The in-transaction principal check precedes compare-and-swap and authority writes. Revocation visible before the check causes zero-write rejection; a later overlapping revocation does not cancel an in-flight commit. This is commit-entry revalidation. Supplement S2 states the policy-lock and stronger-revocation boundaries.

Table 2: Admission properties and their operational interpretation.

Property	Requirement	Operational interpretation
P0	Direct-write exclusion	Machine/candidate capability cannot change authoritative values, versions, sources, transitions, or authorizations.
P1	Candidate non-substitutability	Authority refers to the same canonical persisted certificate, not merely an equal value or analogous candidate.
P2	Context-bound admission	Record, field, persisted evidence content and locator, and expected version agree across the item and certificate.
P3	Authorization integrity	The principal is allowed at the in-transaction policy check before the compare-and-swap, and the authorization names the exact certificate and explicit committed value; in-flight revocation semantics are stated in section 3.5.
P4	Freshness	The expected version equals the transactionally observed current version; stale or competing decisions fail closed.
P5	Successor integrity	One atomic successor has an item-transition bijection, exact changed-field effects, and complete changed/unchanged framing.
P6	Trace soundness	Each committed field resolves through its exact source transition to consistent authorization, certificate, evidence, producer, and version relations.

4. Bounded relational characterization

The Alloy model represents records, fields, ordered versions, evidence, candidates, certificates, principals, authorizations, transitions, and states. Once a record is authoritative, its value and source maps cover exactly its field domain. `MachineEvent` can extend candidate/evidence sets while framing authority (P0). `BatchAdmissionEvent` has a nonempty item set in one envelope. `TamperEvent` freezes values and source pointers while modifying transitions.

The base effect advances one version and applies item values and sources. Full adds unique fields, complete snapshot shape, preexisting certificate/evidence, contextual and authorization bindings, freshness, and an exact item–transition bijection. Separating these conjuncts permits controlled removal. Each ablation uses at least two items: one violates only the selected conjunct and another remains Full-valid. The reduced relation admits that set; Full rejects it with authoritative-state stutter.

Six legal commands exercise Accept, Correction, mixed batches, same-value admission, initial coverage, and singleton admission. Eleven ablations test encoded bindings. Three source attacks remove, replace, or duplicate an anchor after a legal prefix. Fourteen assertions per profile check framing, preservation, singleton correspondence, and trace conditions. The class-to-conjunct map and complete command descriptions are in Supplement S2.

The positive trace assertion requires a trace-complete predecessor and no stray successor transition. It checks admitted fields for one encoded step. Reachable initial and carry-forward witnesses establish a nonempty premise; removing post-state certificate binding changes UNSAT to SAT. Concrete tracing additionally validates predecessor prefixes. This distinguishes bounded preservation from the observation-level proposition.

Two scope profiles increase representative bounds from two records, three fields, six versions, and four states/events to three records, four fields, eight versions, and five states/events. Individual domain bounds remain fixed by deposited commands. SAT denotes a found witness; UNSAT denotes no counterexample within that command’s finite scope (Jackson, 2002; Torlak and Jackson, 2007).

Table 3: Bounded Alloy outcomes in the two declared scopes. SAT denotes a found witness; UNSAT denotes no instance within the encoded scope. Trace SAT counts witnesses to the antecedent of the conditional positive trace assertion.

Profile	Legal SAT	Ablation SAT	Attack SAT	Trace SAT	Total SAT	Total UNSAT	Commands
S1	6	11	3	2	22	14	36
S2	6	11	3	2	22	14	36

5. Transactional realization

5.1. Persistent objects and admission

AUTO-DECTE implements the relation in Python, SQLAlchemy, and SQLite. `CandidateWritePort` appends candidates/evidence, `AuthorityReadPort` loads persisted objects, and `FactAdmissionPort` admits a complete bundle. Recognition receives the candidate facade; trusted review receives admission. This separates capabilities within the process while retaining the host trust boundary.

A certificate retains target, canonical proposal, producer/version, selection artifact, expected fact version, evidence hash, and canonical locator. A decision names reviewer and candidate; its immutable binding fixes the exact certificate and authorized value. Transitions preserve this chain for Accept and Correction. The revocable principal-role mapping is checked inside the transaction.

Admission reloads evidence and reconstructs its canonical locator, checking ownership, content hash, and record/field location against the certificate. The adapter repeats authoritative preflight checks transactionally:

1. reload the field domain, current version, complete predecessor, candidates, evidence, and principal policy;
2. derive initial coverage or later changed fields using canonical JSON equality, rejecting duplicate, hidden, empty, or incomplete effects;
3. validate unused identities and the decision–binding–certificate–evidence–transition relation, including agreement of authorized, transition, and successor values;
4. compare-and-swap the expected current version;

5. create one transition per admitted field, copy exact unchanged sources, and commit decision, binding, complete version, projections, and audit together.

CAS is the first authority write. Failure before commit rolls back admission. Unique indexes and foreign keys reinforce bindings and transition uniqueness. The zero-write baseline is immediately before admission, after any candidate or review preparation. Connection parameters and locator rules are in Supplement S1 and the artifact.

The confirmation API receives candidate, value, and principal from its caller and constructs the binding at confirmation. E2 evaluates an earlier server-held review binding. Manual-source compatibility derives a certificate at confirmation and is outside the preexisting-machine-candidate claim. The historical restricted agent interface permits proposal and verification, with confirmation retained by the host (Supplement S5).

5.2. Reverse trace and source evolution

Tracing checks each field in every prefix version. Initial and changed fields need their unique transition; unchanged fields retain the exact prior source. The verifier checks source record/field/version, committed value, decision, authorization, certificate, producer, evidence hash/locator, and expected-version relations.

A trace is **complete** when these relations agree, otherwise **incomplete** with reasons. It does not repair or substitute sources; legacy pre-certificate states are identified separately. Completeness concerns consistency of persisted authorization lineage under the trust boundary in section 3.

6. Formal–concrete projection and executable conformance

A test-side abstraction connects selected persisted events to the model:

$$\alpha : \text{PersistedPrePost} \rightarrow \text{BatchAlloyInstance}. \quad (19)$$

It independently reads targets, field domains, evidence/locators, certificates, decision/binding rows, version values/sources, transition sets, and pre/post identity sets. Canonical JSON classes become Value atoms, certificates become primitive identities, and a decision/binding becomes Authorization. The complete mapping is in Supplement S2.

A committed wrapper fixes the instance and checks `legalAdmission[event, Full]`. A rejected wrapper requires equal full-database digests, state stutter, and failure of Full. Twenty single-dimension mapping mutants exercise context, freshness, authorization, preexistence, transition, and coverage sensitivity. The adapter shares neither the production planner nor the trace builder.

A second oracle reads all SQLite application tables and checks domains, source evolution, transitions, version anchors, lineage, and values. Five relation mutants exercise its sensitivity. The 35-case catalogue includes five legal cases and thirty rejection, corruption, schema-prevention, stateful-evolution, or oracle-mutant cases. Rejections require the expected public error and unchanged deterministic database digest; post-persistence corruption instead requires a changed digest and trace/oracle diagnosis.

Thirteen regressions test canonical equality, including unchanged controls, numeric/Boolean and nested changes, and no-op rejection. Seven fail before the equality repair and all pass afterward. A separate witness-domain repair retains the five class pairs and adds identity and copy-forward controls (Supplement S2).

Nine intended projections cover singleton and multi-field Accept and Correction, mixed batches, initial coverage, copy-forward, staleness, and whole-batch rejection. They provide selected bounded correspondence. Three scope differences remain explicit: concrete post-initial no-ops are rejected; SQLite uniqueness prevents one modeled duplicate-source state; and concrete prefix tracing checks corruptions and predecessor relations beyond the formal single-level anchor predicate. Thus the positive Alloy assertion concerns one encoded admission step under its well-formed-pre-state condition.

7. Evaluation protocol

7.1. Questions and evidence layers

- RQ1** Do the formal and concrete checks support the admission contract, and what changes when a rich event journal records the exact reviewed candidate?
- RQ2** Which inconsistencies between recorded review intent and confirmation can server-held session binding reject, and which remain outside its guarantee?
- RQ3** What admission, reverse-trace, and storage costs occur on the repaired implementation under the declared workloads?

The retained Alloy, projection, and 35-case catalogue evidence is distinguished from E1–E3, which use the equality-repaired reference at commit `c6d512843c905cab6d8521dd8c914f7fb26d85ae`. Before the new studies, 85 selected implementation regressions and ten harness checks were rerun. The prospective local protocol, pilot amendments, environment, raw receipts, and execution status are documented in Supplements S1 and S6. E1–E3 make no hosted-model calls; historical agent runs remain separate in S5.

Figure 2 connects these evidence layers to the questions they answer. Constructed execution, provenance-answer, browser-case, and timing denominators remain separate.

7.2. *RQ1: rich-context and exact-binding comparison*

A separately written `sqlite3` journal retains immutable candidates/events, review context, versions, transactions, and complete field sources. Its *context* configuration checks field, version, evidence, and retained context while omitting candidate/certificate identifiers and instance-creation time. Its *exact* configuration adds the reviewed-candidate link. The third arm invokes the reference’s actual transaction. We score both instance authorization and the looser value/context policy.

Fifteen inputs cross eleven case families and three mechanisms, giving $15 \times 11 \times 3 = 495$ executions. Twelve deterministically selected archived proposal cells contain four distinct values; three synthetic type controls bring the total to seven. They enter newly constructed histories, rather than complete historical-task replays. Cases exercise Accept, Correction, multi-field admission, equal-valued candidate and different-evidence substitutions, staleness, replay, unauthorized value, and three interrupted transaction locations. Supplement S6 lists the inputs and cases.

After acceptance, an independent reader checks six provenance questions on three fields: proposal, authorized value, reviewer, reviewed candidate, current value, and source version. Each accepted history contributes eighteen answers, classified as correct, wrong, ambiguous, or unavailable. Rejection requires the expected reason and unchanged logical fact-database digest, measured after authorization preparation. Prior review records may persist. Original strengthened-baseline continuity checks have a separate denominator.

7.3. *RQ2: review-to-confirmation boundary*

Local Chrome/Playwright runs an instrumented review fixture backed by `ReviewForms.confirm`. An independent driver records the displayed

From admission relation to executable evidence

Each evidence layer answers a different question.
Constructed cases and timed calls keep separate denominators.

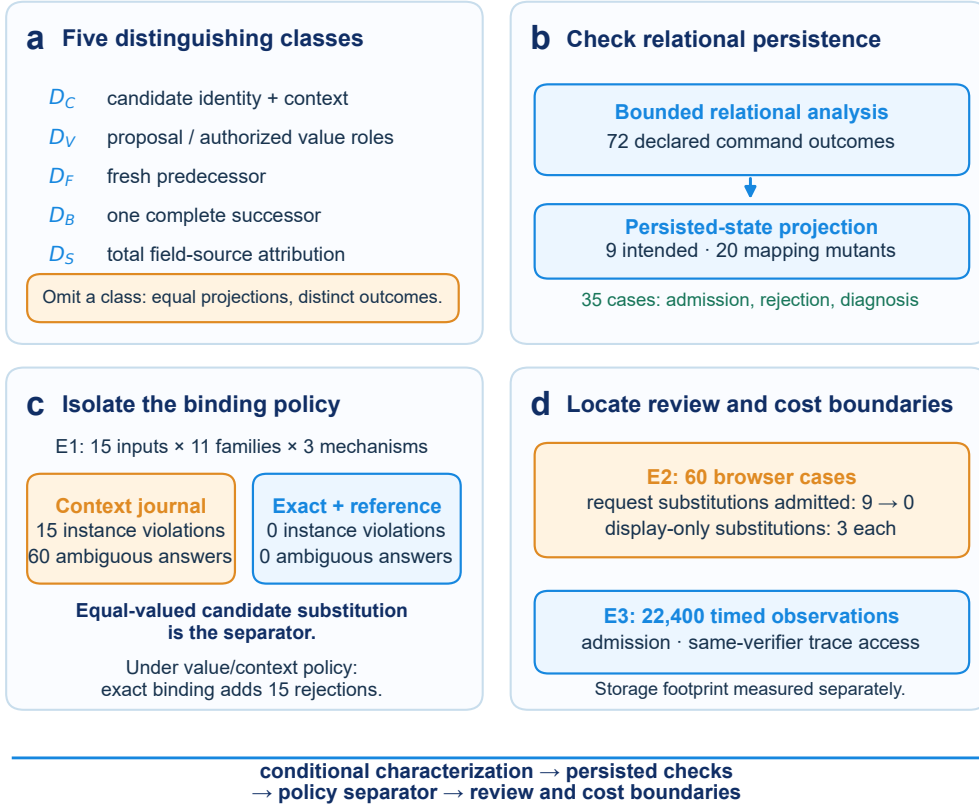


Figure 2: Evidence route from the conditional characterization to persistence checks and E1–E3. E1 compares instance and value/context policies; E2 compares the original and session-bound confirmation paths; E3 measures call and access costs. E1/E2 counts and E3 timing totals are read from the archived tables and CSV summaries. OpenAI Codex (GPT-6, 27 September 2026) assisted diagram scripting; Matplotlib 3.10.8 rendered the deterministic vectors.

candidate, proposal, and authorized value before confirmation. Three JSON inputs cross ten interactions and two paths, giving sixty cases; six confirmations separately prepare replay setups. Positive controls cover acceptance, correction, and reselection. Candidate/value/principal request substitutions, stale/replayed requests, interrupted commits, and display-only substitution probe distinct boundaries.

The original path is compared with a server-held session binding candidate, version, reviewer, and value. Acceptance, agreement with recorded review intent, and writes on rejection are recorded. Fixed principal labels and a separate session database make this a scripted boundary study; authentication, human attention, and cross-database crash atomicity remain outside its evidence.

7.4. RQ3: current-version costs

Two ten-cell admission comparisons contribute 4,000 observations each; a 36-cell trace comparison contributes 14,400. Each cell has five warmups and 200 timed calls per arm in seeded randomized order, for 22,400 timed observations excluding warmups. These measurements come from one machine with uncontrolled background activity; environment versions and full grids are in Supplements S1, S4, and S6.

The mechanism comparison times complete confirmation on shared recognition-source workloads. Construction and database cloning are excluded: the journal opens its connection before timing, whereas the reference may connect lazily inside confirmation. Schema, ORM, validation, and application functionality also differ. These are costs at the declared call boundary, rather than isolated binding overhead.

A separate manual-source materialization ablation times persistence of a prevalidated delta while retaining authority rows, complete sources, transactionality, and CAS. Its result must match the precomputed relational fingerprint. Validation and planning occur outside that timer; its full-path column belongs to a different workload from the mechanism comparison.

The trace comparison holds the verifier fixed and changes three bulk repository reads to identifier enumeration and point lookups. Every returned complete object must agree. The grid varies record width, history length, and total records. Storage fixtures populate full/reduced schemas at three transition scales; their footprint excludes external evidence/model outputs, and the reduced schema has fewer audit obligations.

8. Results

8.1. RQ1: exact binding and persisted conformance

E1: equal-valued candidate separation. All 495 executions completed without harness errors. The exact journal and reference agreed on all 165 paired decisions and query-answer sets. Each accepted 45 legal cases and rejected 120 without fact-database writes after authorization preparation, including all three rollback locations.

The context journal accepted 15 equal-valued substitutions. They satisfy its value/context policy but violate instance authorization; the exact mechanisms therefore make 15 additional rejections under the looser policy. All mechanisms reject substitutions with different retained evidence. Equal-valued candidate substitution isolates what richer context alone still leaves unresolved.

Accepted context-journal histories produced 1,020 correct answers and sixty ambiguous reviewed-candidate answers. Each exact mechanism produced 810 correct answers and none wrong, ambiguous, or unavailable. Totals differ because each accepted history contributes eighteen answers. These observations concern the constructed histories, not independent query workloads.

The original strengthened-baseline continuity run reproduced seven-of-eight single-history agreement, with equal-valued substitution as the separator; its paired-history distinction is a separate ninth check. The richer exact journal closes that gap in E1 by preserving the authorization link. The evaluated relation is therefore enforceable in both storage designs.

Formal and persisted support. All 72 Alloy outcomes matched the manifests: each profile had six legal, eleven ablation, three source-attack, and

Table 4: E1: 165 executions per mechanism. Instance violations are accepted cases violating the instance policy. Extra rejections are rejected cases valid under the value/context policy. Query counts refer to field-level answers after acceptance. All arms had zero value/context-policy violations and zero rejections with fact writes.

Mechanism	Accepted	Instance	Extra	Query answers	
	cases	violations	rejections	Correct	Ambiguous
Context journal	60	15	0	1,020	60
Exact journal	45	0	15	810	0
Reference	45	0	15	810	0

two nonempty-precondition witnesses, plus fourteen assertions without counterexamples within scope. The projection produced nine satisfiable intended instances and twenty unsatisfiable mapping mutants. All 35 catalogue cases, two stateful profiles, and 33 rollback/concurrency tests passed in the retained execution.

The illustrative lifecycle retained proposal 100, separately authorized correction 101, exact sources for the other two fields, and a complete trace/export. Six archived AI-origin lifecycles and ten pinned-interface controls also matched their outcomes. The new execution passed 85 selected regressions and ten harness checks. These are separate evidence layers; the retained tests were not all rerun for E1–E3.

8.2. *RQ2: request binding and display controls*

Each path completed thirty browser cases and accepted all nine legitimate controls. The original path accepted nine request substitutions inconsistent with recorded review intent; the server-held gate rejected all nine. Both rejected stale submissions, replays, and precommit interruptions without fact writes.

Both nevertheless accepted three display-only substitutions: the page showed B while server and request continued with A. The original transaction preserves the caller-supplied binding; the gate adds consistency with an earlier server-held review record. Faithful presentation remains a premise for both.

8.3. *RQ3: admission, trace, and storage costs*

Admission. Both admission comparisons completed ten cells. Shared mechanism queries agreed; materialized states matched their precomputed fin-

Table 5: E2: accepted cases by interaction family, with three JSON values per named interaction. Each path has thirty cases; replay setup is additional. All nine original-path rejections and eighteen gate rejections made zero fact writes.

Interaction family	Cases/path	Original	Session gate
Accept, Correction, reselection	9	9	9
Candidate, value, principal request substitutions	9	9	0
Stale, replay, precommit interruption	9	0	0
DOM-only substitution	3	3	3
All cases	30	21	12

gerprints. Table 6 shows representative cells; Supplement S4 retains every cell.

With 128 fields all changed, median reference confirmation was 712.00 ms and exact-journal confirmation 22.09 ms. Both enforce exact binding; the difference describes the implementation and connection boundaries in section 7. In the separate manual-source ablation, full confirmation was 427.59 ms and prevalidated materialization 23.50 ms. The latter excludes validation and planning from timing and is not a complete admission path.

Reverse trace. All 36 paired cells completed 14,400 calls with identical full returned objects. At 128 fields, 100 versions, and 1,000 records, bulk p50/p95 was 170.24/216.75 ms versus 509.09/595.85 ms for point access, a median ratio of 2.99.

Bulk tracing used twelve SQL statements in every measured cell; the largest point-access cell used 693. Both arms run the same current verifier, so the difference concerns repository access. Processing, prefix traversal, and memory can still grow with history and fields despite constant round trips.

Schema footprint. Full-schema sizes were 1.52, 13.16, and 134.53 MiB at the tested transition scales, with roughly 1.1 kB incremental main-database storage per transition relative to the reduced fixture.

These synthetic fixtures exclude external evidence and model-output files. The reduced schema has fewer audit obligations; the measurements characterize their footprints.

8.4. Separate historical integration

The archived matrix retains 1,260 planned runs. Strict benign yield was 320/360 (88.89%) and post-hoc endpoint yield 331/360 (91.94%), with 25

Table 6: Representative E3 admission cells: p50/p95 (ms), 200 observations per arm/cell. Recognition and manual-source columns use separate workloads; prevalidated timing excludes validation and planning. All ten cells are in Supplement S4.

Fields	Changed	Recognition-source mechanisms		Manual-source ablation	
		Reference	Exact journal	Full	Prevalidated
1	1	28.79/50.50	10.40/13.32	23.23/33.76	9.97/14.63
32	32	187.24/292.72	13.07/17.77	122.68/200.00	13.89/22.61
128	128	712.00/1039.83	22.09/30.53	427.59/621.20	23.50/37.43

Table 7: E3 trace: representative cells with 128 fields and 1,000 total records. Latencies are p50/p95 milliseconds; each arm has 200 timed observations per row. SQL counts are constant within each row and arm. The complete 36-cell grid is deposited.

Versions	Latency (ms)		SQL statements	
	Bulk	Point reads	Bulk	Point reads
1	28.68/43.26	209.81/256.03	12	396
10	39.69/48.16	233.82/285.43	12	423
100	170.24/216.75	509.09/595.85	12	693

Table 8: E3 synthetic schema footprint after checkpointing. Incremental bytes per transition equal the full-minus-reduced main-database size divided by transition count. External evidence files are excluded.

Transitions	Full (MiB)	Reduced (MiB)	Incremental bytes/transition
1,000	1.52	0.48	1085.4
10,000	13.16	2.88	1077.2
100,000	134.53	28.45	1112.3

unscorable runs; all 93 runtime failures remain recorded. Fixed host checks recorded no unauthorized mutation in 720 invalid-tuple calls or 179 capability-unavailable branches, the latter making no admission call. Supplement S5 preserves conditional denominators, configuration-specific failures, and annotation. These support restricted-interface integration separately from E1/E2.

9. Discussion and limitations

9.1. Correction-aware authority as a data relation

The result links three roles that a final value cannot recover: proposal origin, human value authority, and committed field source. Admission binds these roles to one candidate and fresh predecessor while constructing a complete successor. The paired histories explain the corresponding observation distinctions; the formal, projection, and transaction checks test their enforcement.

The journal comparison locates the contribution in this relation. Exact binding gives the journal and reference the same 165 decisions and

query-answer sets. Instance accountability requires rejecting the 15 context-equivalent substitutions; value/context accountability permits them. Applications should choose the granularity that their audit questions require. Renewed approval without a value change belongs to a separate review-event path, because concrete post-initial admission creates transitions only for changed values.

9.2. Trust and validity boundaries

The server-held gate demonstrates request-to-review consistency for the tested substitutions. Display-only controls retain a separate presentation boundary. Host authentication, review-channel integrity, principal policy, hashing/canonicalization, and database execution remain trusted. Recorded lineage identifies authorization and sources, not factual truth. Evidence checks compare stored hashes and locators rather than re-reading external bytes; unnoticed external replacement requires immutable storage or additional integrity checks. Commit-entry policy revalidation does not cancel later-revoked in-flight operations (Supplement S2).

Proposition 1 concerns the declared projections, opaque handles, and failure model. Alloy checks finite scopes; selected projections establish bounded correspondence. The formal no-op, schema-uniqueness, and trace-prefix differences are stated in section 6. Distributed revocation, other database engines, deletion/redaction, and retention expiry require additional design and evidence.

E1 reuses seven distinct proposal values in constructed cases; the journals are same-study implementations. E2 uses a local scripted fixture, fixed principals, and separate session/fact databases. Its evidence does not extend to human attention, hostile hosts, or cross-database crash atomicity. Historical hosted-agent outcomes and author-involved annotation have separate scoring, missingness, and failure records in Supplement S5.

E3 characterizes one machine with uncontrolled background activity. The admission implementations differ in schema, ORM, validation, features, and connection lifecycle; the materialization timer omits validation/planning. The paired trace study holds the verifier fixed, identifying an access-pattern difference rather than total cost independent of history. Complete workload and validity details are in Supplements S4, S6, and S7.

9.3. Applying the relation

Adoption begins with the approval policy and one correction traced through persisted records. Identify the reviewed candidate, authorized value, predecessor check, and changed/unchanged source links. Retain a legal control and a failure probe for each boundary. Supplement S3 provides the worked example and inspection checklist. An independently implemented deployment with operational reviewers would provide the next test of transferability.

10. Conclusion

Correction-aware admission links an exact persisted candidate, explicit human-authorized value, fresh predecessor, and complete successor with total field sources. Five paired histories establish class-wise irredundancy within the declared observation model. Bounded relational analysis, selected persisted-state projections, and transaction tests connect the relation to an executable realization.

The exact-binding event journal and reference agree on all 165 corresponding constructed cases; equal-valued substitution separates their decisions from context-only approval. A review-session gate rejects the tested request substitutions while display integrity remains assumed. Current-version workloads characterize admission and source reconstruction, including bulk/point tracing with the same verifier. Together, these results make the correction-to-authority relation enforceable at commit and queryable afterward under the stated trust and workload boundaries.

Declarations

Data and software availability

The study uses synthetic and public lifecycle inputs. The new comparison, browser-boundary, and current-version cost evidence is deposited with code, raw receipts, database snapshots, screenshots, dependency locks, and manifests at <https://github.com/lhh666-6/auto-dete/tree/2645e5e18c900ea91c9c980e44195dc71e410432/DKE-supplement>. The reference implementation is fixed at commit `c6d512843c905cab6d8521dd8c914f7fb26d85ae`, under `latest/code/implementation-fixed`. Its formal models, observation witnesses, and earlier annotation and integration evidence are accessible from <https://github.com/lhh666-6/auto-dete/tree/r31-jss-2026-09-13/latest>. Historical v8 evidence remains separately versioned under tag

r21-jss-2026-09-07-v8. Supplement S1 maps the evidence layers and describes restoration of the losslessly compressed large database. Inspecting the deposit and reproducing the three new experiments requires no paid model calls; repeating historical hosted-agent runs requires provider credentials.

Ethics approval and consent to participate

The new experiments use constructed data and scripted browser interactions, with no new human participants. In the historical annotation study, one author (X.W.) and one non-author volunteer independently coded benign completion outcomes; L.H. adjudicated the inter-human disagreement and separately reviewed eleven human–rule disagreements against the strict-trajectory criterion. X.W. also completed the textual-acknowledgment annotation. Only pseudonymous annotator identifiers were collected; both benign-endpoint coders were informed of the task and data use, and the non-author coder participated voluntarily. Annotation evaluated generated outputs rather than annotator characteristics. The authors assessed this activity as not requiring ethics approval. No animal subjects were involved.

CRedit authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing–original draft, and Writing–review & editing.

Xuan Wentao: Methodology, Formal analysis, Investigation, Data curation, Validation, Visualization, and Writing–review & editing.

Chen Qile: Investigation and Validation.

Peng Peng: Supervision and Writing–review & editing.

Funding

This research received no external funding.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI and AI-assisted technologies

DeepSeek, OpenAI Codex, and Claude assisted earlier stages of code, analysis, figure, and manuscript preparation. OpenAI Codex assisted the present revision with supplementary experiment implementation and execution, evidence checks, literature organization, manuscript restructuring and drafting, and deterministic table generation and PDF compilation. During the 27 September 2026 editorial revision, OpenAI Codex (GPT-6) assisted compression and vector-diagram scripting. OpenAI’s image-generation tool produced a conceptual design prototype from an author-supplied style reference; its model/version identifier was not exposed. The prototype is archived separately and is not a manuscript figure. The final conceptual diagrams use checked vectors rendered with Matplotlib 3.10.8; experimental counts are read from deposited tables and CSV summaries. The three new experiments use archived proposal values and synthetic inputs and make no hosted-model calls. Historical benchmark calls to two requested OpenAI configurations and one requested DeepSeek configuration are distinct research data, with requested identifiers and raw events retained in the archive. Historical human annotation labels were supplied by their named human coders; AI review proposals do not replace those labels or constitute human adjudication. No AI-generated raster figure is included. Research interpretation, source verification, approval of the final manuscript, and responsibility for the work rest with the authors.

References

- Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2018. Using reenactment to retroactively capture provenance for transactions. *IEEE Transactions on Knowledge and Data Engineering* 30, 599–612. URL: <https://doi.org/10.1109/TKDE.2017.2769056>, doi:10.1109/TKDE.2017.2769056.
- Buneman, P., Khanna, S., Tan, W.C., 2001. Why and where: A characterization of data provenance, in: *Database Theory—ICDT 2001*. Springer Berlin Heidelberg. volume 1973 of *Lecture Notes in Computer Science*, pp. 316–330. URL: https://doi.org/10.1007/3-540-44503-X_20, doi:10.1007/3-540-44503-X_20.
- Cheney, J., Chiticariu, L., Tan, W.C., 2009. Provenance in databases: Why,

- how, and where. *Foundations and Trends in Databases* 1, 379–474. URL: <https://doi.org/10.1561/19000000006>, doi:10.1561/19000000006.
- Cui, H., Tang, Z., Yao, Z., Meng, F., Ma, Q., Jia, W., 2026. MemTxn: A transaction boundary for source-supported updates and complete-state recovery in agent memory. URL: <https://arxiv.org/abs/2607.27834>, doi:10.48550/arXiv.2607.27834, arXiv:2607.27834. preprint.
- Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramèr, F., 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: *Advances in Neural Information Processing Systems*, Neural Information Processing Systems Foundation. pp. 82895–82920. doi:10.52202/079017–2636.
- Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., Waite, D., 2023. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/rfc9449/>, doi:10.17487/RFC9449.
- Fowler, M., 2005. Event sourcing. Architecture pattern article. URL: <https://martinfowler.com/eaaDev/EventSourcing.html>. published 12 December 2005; accessed 26 September 2026.
- Gray, J., 1981. The transaction concept: Virtues and limitations, in: *Proceedings of the Seventh International Conference on Very Large Data Bases*, pp. 144–154. URL: <https://web.stanford.edu/class/archive/cs/cs240/cs240.1236/old/sp2014/readings/Gray81.pdf>.
- He, J., Yu, D., 2026a. Beyond memory: A transactional continuity kernel for long-lived AI agents. URL: <https://arxiv.org/abs/2608.11632>, doi:10.48550/arXiv.2608.11632, arXiv:2608.11632. preprint.
- He, J., Yu, D., 2026b. Stored is not supported: Typed provenance and assertion guardrails for persistent AI agents. URL: <https://arxiv.org/abs/2609.02127v1>, doi:10.48550/arXiv.2609.02127, arXiv:2609.02127. preprint, version 1, 2 September 2026.
- Jackson, D., 2002. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology* 11, 256–290. URL: <https://doi.org/10.1145/505145.505149>, doi:10.1145/505145.505149.

- Kung, H.T., Robinson, J.T., 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 213–226. URL: <https://www.eecs.harvard.edu/~htk/publication/1981-tods-kung-robinson.pdf>, doi:10.1145/319566.319567.
- Marinov, D., Khurshid, S., 2001. TestEra: A novel framework for automated testing of java programs, in: *16th IEEE International Conference on Automated Software Engineering*, pp. 22–31. URL: <https://doi.org/10.1109/ASE.2001.989787>, doi:10.1109/ASE.2001.989787.
- Moreau, L., Missier, P., 2013. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium. URL: <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.
- Saidi, W., 2026. MutMem: Cryptographically authorized mutation in persistent agent memory. URL: <https://arxiv.org/abs/2608.02843>, doi:10.48550/arXiv.2608.02843, arXiv:2608.02843. preprint.
- Torlak, E., Jackson, D., 2007. Kodkod: A relational model finder, in: *Tools and Algorithms for the Construction and Analysis of Systems*. Springer. volume 4424, pp. 632–647. URL: https://doi.org/10.1007/978-3-540-71209-1_49, doi:10.1007/978-3-540-71209-1_49.
- Zhou, H., Zhang, L., Fan, Y., He, T., Chen, S., Geng, H., Torr, P., Yin, Z., 2026. LatticeMind: A conflict-aware memory primitive for multi-agent systems. URL: <https://arxiv.org/abs/2608.08236>, doi:10.48550/arXiv.2608.08236, arXiv:2608.08236. preprint.